

shm 信頼性強化マージ提案

対象: shared-memory-based-handy-communication-manager / react_cv / sensor_daemons ブランチ: add-race-condition-regression-tests → main コミット数: 143 (shm 94 / sensor_daemons 27 / react_cv 22) 作成: 2026-09-03

独立レビュー 5 巡と共通化リファクタリングによる、製品組込みに向けた信頼性強化です。共有メモリのワイヤ形式が変わるため、**全プロセスの停止を伴う移行**が必要です。

判断

マージしてよい。ただし Raspberry Pi 4 での全体ビルド再確認と、移行手順に沿った一斉再起動が前提。

利用側のソースコードは原則そのままビルドできます (main 時代の典型コードが無修正で動くことを実測確認済み)。一方で共有メモリのワイヤ形式に互換性はありません。旧セグメントが残ったまま新バイナリを起動すると接続に失敗します。

前提条件

- C1 Pi4 (aarch64) で全体ビルドとテストを再確認する。前回確認は 9/2 で、以降 33 コミットが入っている
- C2 移行時に `shm_tool remove` で旧セグメントを消し、そのトピックを使う**全プロセスを再起動**する
- C3 `std::string` / `std::vector` をメンバに持つペイロード型があれば、事前に洗い出す (コンパイルエラーになる)

数字で見る変更

項目	値	補足
コミット	143	shm 94 / sensor_daemons 27 / react_cv 22
実装の差分	+11,813 / -3,880 行	37 ファイル
テスト	165 件	main 時代のテストファイル 4 → 22
レビュー	5 巡	記録 12 本を review/ に同梱
ABI	v1 → 4	ワイヤ形式は非互換

この変更は何のためか

目的は 2 つ。主目的が信頼性、副目的が新機能の試験導入です。

主目的 — 製品組込みに耐える信頼性

独立したレビューを5巡かけ、見つかった不具合を回帰テストとともに潰しました。実際に発見・修正されたものには、プロセスを SIGSEGV / SIGABRT で落とす欠陥、センサ値が黙って消える欠陥、別時刻のセンサ値を「同時刻」として返す欠陥が含まれます。いずれも実機で踏み得るもので、テストで再現してから直しています。

副目的 — タイムマシン機能の試験導入

「いま更新されたオドメトリと同じ時刻のスキャンが欲しい」を直接書ける API を追加しました。既存 API には影響しないので、使わなければ従来どおりです。

API	用途
subscribeAlignedTo()	別トピックのサンプルに時刻を合わせて読む。ずれの上限を必須引数で指定
subscribeAt()	時刻を直接指定して読む（Nearest / AtOrBefore / AtOrAfter）
getRetentionWindow()	いま引ける時刻の範囲を確認する
subscribe(state, info)	発行番号・取得時刻・長さを SampleInfo で受け取る

検索に使う時刻は CLOCK_MONOTONIC_RAW のみです。壁時計は記録しますが、NTP 補正で前後に飛ぶため基準にしません。保持できる履歴の長さは buffer_num ÷ 発行レート なので、10Hz のセンサを buffer_num = 3 で持つと 300ms しか遡れません。

影響範囲

「自分は何をすればよいか」を立場別に示します。

立場	影響	必要な対応
パイロード型を定義している	要確認	std::string / std::vector をメンバに持つ型はコンパイルエラーになる
Publisher / Subscriber を使っている	原則なし	ソース変更は不要。再ビルドと再起動のみ
LiDAR / 点群を publish している	挙動変化	失敗時に例外を投げるようになる（従来は std::cerr のみ）
センサ daemon を運用している	改善のみ	対応済み。失敗を数えて継続し SensorStatus に反映する
shm_service / shm_action を使っている	削除	該当なしを確認済み（利用箇所ゼロ）
運用・デブロイ	手順変更	旧セグメントの削除と全プロセス再起動が必須

詳細 — なぜソース変更が原則不要と言えるのか

Publisher / Subscriber の既存シグネチャは1つも変わっていません。追加のみです。main 時代の典型的な利用コードをそのまま書いてビルド・実行し、確認しました。

```
Publisher<Pose> pub("legacy_pose", 3);
Subscriber<Pose> sub("legacy_pose");
sub.setDataExpiryTime_us(2000000);
pub.publish(Pose{1,2,3});
const Pose &p = sub.subscribe(&ok);
sub.existsPublisherMemory(); sub.waitFor(1000);
```

```
→ scalar : ok=1 x=1 existsPublisherMemory=1
→ vector : ok=1 size=3
```

`std::vector<T>` 特殊化も同様です。したがって利用側で必要なのは**再ビルド**だけで、コードの書き換えは下記3つの例外を除いて発生しません。

詳細 — 破壊的変更 1: 型制約が厳しくなる（コンパイルエラー）

`SHM_STRICT_TYPE_CHECK` が既定 ON になり、共有メモリに置けない型が**コンパイル時に**弾かれます。main には検査自体がありませんでした。

```
struct HasString { int a; std::string s; };
irlab::shm::Publisher<HasString> p("x");

error: static assertion failed: shm::Publisher : payload type must be
trivially copyable to live in shared memory. ... Members such as
std::string / std::vector hold pointers that are meaningless in
another process.
```

これは**従来なら黙って壊れていたもの**です。共有メモリはバイト列なので、内部にポインタを持つ型を置くと他プロセスでは無意味なアドレスを読むことになります。エラーになる型が見つかった場合、そのトピックは以前から正しく動いていなかった可能性が高く、修正の機会と捉えるべきです。

ワークスペースには**147 種類**のペイロード型があります。`-DSHM_STRICT_TYPE_CHECK=OFF` で一時的に従来の実行時検査だけに戻せますが、ARM でのみ働く非対称な検査になるため**恒久的な回避策としては推奨しません**。

詳細 — 破壊的変更 2: `publish` の失敗が例外になる（LiDAR / 点群）

`Lidar2dScanData` と `PointCloud2DScanData` の `publish()` は、従来「失敗しても `std::cerr` に出して続行、戻り値 `void`」でした。これは**呼び出し側から取りこぼしを知る手段が無い**ということで、全スロットが購読側に押さえられて1スキャンも書けなくても、daemon はログを吐きながら「成功」として回り続けていました。

他の型（POD / `std::vector` / `cv::Mat`）は以前から例外を投げていたため、**同じ構成の daemon でも型によって落ちたり落ちなかったりする状態**でした。5 つとも例外に統一しています。

コンパイルは壊れません。`pub.publish(x)` は戻り値の有無に関わらず通り、変わるのは実行時の挙動だけです。ワークスペース全体を走査した結果は次のとおりです。

分類	箇所	状態
<code>try</code> で覆われている	7	影響なし
<code>try</code> 無し・ <code>cv::Mat</code> （元から例外）	5	挙動不変
<code>try</code> 無し・今回から例外	3	対応判断済み

今回から例外が飛ぶ3箇所は、性質ごとに判断しました。

- `synthetic_line_publisher` — 常駐して発行し続けるツール。**修正済み**（数えて継続）。修正前は `std::terminate` でプロセス消滅していた
- `sim_test.cpp` — 決定論的なオフライン試験。**そのまま**（publishに失敗したなら結果が無効で、続行する方が有害）
- `basic_usage_example.cpp` — サンプル。**そのまま**（失敗すれば例外が飛ぶことを示すのが正しい）

詳細 — 破壊的変更 3: ワイヤ形式が非互換（最重要）

運用上の必須事項 旧セグメントを消さずに新バイナリを起動すると、接続に失敗します。新旧のプロセスが混在した状態は動きません。

mainの共有メモリ形式はv1（先頭が初期化フラグ、ABI_MAJORという概念自体が無い）でした。本ブランチは形式v3/ABI 4で、先頭に192バイトの`ShmHeader`を置き、magic・ABI版・レイアウト・型の取り決め・世代タグを記録します。

magic値は'SHM2'で、v1の先頭4バイト（初期化フラグ0/1/2）とは必ず不一致になります。したがって**新しいコードが古い領域を誤って読むことはありません**——接続を拒否します。`shm_tool doctor`は該当セグメントを「別形式またはshm v1 → `shm_tool remove`が必要」と報告します。

1トピックは `/dev/shm/shm_<topic>` と、レイアウト世代ごとの `shm_<topic>#<世代>-<ノンス>` の組で構成されます。手で前者だけ消すと世代が孤児として残るため、**`shm_tool remove` を使ってください**。

詳細 — 削除したもの（shm_service / shm_action）

要求応答型の `shm_service` と非同期処理型の `shm_action` を削除しました。利用箇所がゼロであることを確認済みです。もし社外・別リポジトリで参照している箇所があれば、マージ前に申告してください。

あわせて、生成物である `docs/`（462 ファイル）をリポジトリから外し、GitHub Actionsで生成してPagesへ配信する形にしました。追跡ファイルの85%が生成物で、`manual/*.md`を1行直すたびに数十ファイルの差分が乗る状態でした。差分の-48,044行はこの削除によるものです。

信頼性として何が直ったか

実機で踏み得る欠陥のうち、代表的なものを挙げます。すべて回帰テストつきです。

症状	原因	由来
プロセスが SIGABRT で落ちる	優先度継承つき robust mutex への待ち時間つきロック (glibc が abort する)	R05
publish 中に SIGSEGV	使用中のマッピングを差し替えていた。shm_tool remove の実行が引き金	R04
センサ値が黙って消える	後発 Publisher が全タイムスタンプを消していた	R02
購読ノードが落ちると最新値も消える	EOWNERDEAD を「書き込み中の死」と誤判定していた	R04
新旧が混ざった値を「成功」で返す	コピー中の上書きを検出していなかった (torn read)	R01
別時刻の値を「整列済み」として返す	payload と時刻情報が別サンプルになる窓があった	R02
センサ時刻が系統的に遅れる	取得時刻ではなく書き込み完了時刻を記録していた (実測 379μs → 1μs)	R06
センサ daemon が突然停止する	publish の例外がループを抜け、スレッドごと終了していた	R06

詳細 — レビューの進め方と、途中で分かったこと

R01 から R05 まで、毎回独立したレビューを掛け、指摘ごとに回帰テストを追加しています。各テストは「修正を巻き戻すと落ちる」ことを確認してから採用しました。

この過程で、テスト自体が機能していない例が 3 件見つかっています。

- **R04** — 世代切替の修正を丸ごと取り消しても全テストが緑のままだった (決定論的なフックを追加して解決)
- **R05** — レイアウトハッシュのテストが、使った 2 型のサイズが違ったため偶然通っていた
- **R06** — 返り値ダブルバッファのテストが、失敗を作る方法のせいでコピー処理に到達していなかった

また R05 では、個別には正しい 2 つの修正が、重なったときだけプロセスを落とすケースが見つかりました。しかも ThreadSanitizer ではスピン待ちにフォールバックするため原理的に検出できず、138/138 が緑のまま潜んでいました。

詳細は review/r01 ~ review/r06 の 12 本にすべて記録してあります。

詳細 — 検証の範囲

構成	結果
Release	165 / 165
AddressSanitizer	164 / 164
UndefinedBehaviorSanitizer	164 / 164
ThreadSanitizer	164 / 164
lidar / point_cloud / cv::Mat 各回帰	6 / 6 × 3
5 特殊化の適合性スイート	11 / 11 × 5
sensor_daemon_base	10 / 10

サニタイザ構成が1件少ないのはPythonバインディングのテストを登録しないためです。テストは `/dev/shm` を共有するため、mount namespace で隔離して実行しています。

適合性スイートは今回の新規追加です。 `Publisher` / `Subscriber` には5つの特殊化があり、型に依存しない約150行が各所にコピーされていました。「5つが守るべき契約」を1箇所書き、全特殊化に同じ11件の検査を掛けています。その後に関通化（`SubscriberCore` / `PublisherCore`）を行い、**1箇所直せば5つ全部に効く状態**にしました。

移行手順

順序に意味があります。とくに3と4を逆にすると、旧セグメントを掴んだまま起動します。

1. Pi4で全体ビルドを再確認する（マージ前）

前回のPi4確認は9/2の時点で、以降33コミットが入っています。とくに `SubscriberCore` / `PublisherCore` による共通化はヘッダを大きく変えるため、x86で通ってもaarch64で落ちる型の問題が起き得ます。

過去の実例 書式宣言の置き場所の誤りは、x86のgcc 11では通り **Pi4の全体ビルドで初めて落ちました**。
「x86で通ったこと」はaarch64で通る根拠になりません。

2. ペイロード型を洗い出す

ワークスペースを一度ビルドし、型制約で止まる箇所を確認します。止まった型は共有メモリに置けない型です。

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc) 2>&1 | grep "must be trivially copyable"
```

該当があれば、固定長配列に置き換えるか、`serialize()` を持つ型として特殊化を用意します。

3. 全プロセスを停止する

そのトピックを使うpublisherとsubscriberを**すべて**止めます。混在状態では動きません。

4. 旧セグメントを削除する

```
shm_tool doctor          # 何が残っているか、どれが古い形式かを確認
shm_tool remove <topic>  # トピックごと（世代セグメント込み）で削除
```

`rm /dev/shm/shm_*` は使わないでください。世代セグメントが孤児として残り、`doctor` が「rootセグメントが無い」と報告する状態になります。

5. 新バイナリで起動し、doctorで確認する

```
shm_tool doctor
# ABI・書式・履歴・異常を一覧。対処が必要なものがあれば終了コードが変わる
```

起動スクリプトの前段に置いて、古いセグメントが残ったまま立ち上がるのを防ぐ使い方ができます。

6. (任意) 書式宣言へ順次移行する

`SHM_DECLARE_LAYOUT()` を付けると、**同じサイズのままメンバを並べ替えた変更**まで検出できます。版番号を手で維持する必要はありません。既定は OFF なので、パッケージ単位で `-DSHM_REQUIRE_LAYOUT=ON` にして順に潰せます。未宣言のトピックは `doctor` が表示します。

詳細 — 移行中に踏みやすい点

- **古い `.so` を掴む** — 共有ライブラリに `lib` 接頭辞が付かないため、`shm_pub_sub.so` は `shm_base.so` を素の名前で参照し `RUNPATH` を持ちません。 `LD_LIBRARY_PATH` の先頭にあるものが勝つので、`ldd` で解決先を確認してください。
- **`subscribe()` の既定期限は 2 秒** — 低レート of トピックで「publish しているのに受信できない」の 最有力原因です。 `setDataExpiryTime_us(0)` で無効にできます。
- **返り値の参照は 2 回目で化ける** — `subscribe()` の返り値は内部バッファへの参照で、1 回は生き残りますが 2 回目で書き換わります。保持するなら値でコピーしてください。
- **トピック名の `/` は `_` に置換** — `"a/b"` と `"a_b"` が無警告で衝突します。

これらは `manual/pitfalls_jp.md` に 12 項目としてまとめてあります (すべて手元で再現してから記載)。

残るリスクと未対応

隠さず記載します。いずれも判断理由を `review/` に残しています。

既知の制約 — 複数 publisher + 容量拡張で 1 件落ち得る

可変長トピック (`std::vector` / スキャン / 点群 / 画像) は容量が増えると新しい世代へ切り替えます。このとき、履歴のスナップショットを取ってから世代を切り替えるまでの間に別の publisher が書き込むと、そのサンプルが引き継がれず、かつ再発行もされません。 `SampleInfo::sequence` に欠番として現れます。

条件は「同じトピックに publisher が 2 つ以上あり、かつ容量拡張が起きる」 ことです。publisher が 1 つなら起きません (拡張を起こすのも書き込むのも同じ呼び出しのため)。

回避策は次のいずれかです。

- 1 トピック 1 publisher にする (最も確実)
- 起動時に最大サイズを一度 publish して容量を先に確保する (容量は増やすだけで運用されるため、以後の拡張が起きない)

塞げないのは、窓を消す手段がどれも別の問題を生むためです。切替後にさらい直すと同じ測定が二重に現れ、スナップショットを取り直しても新世代のスロットは既に埋まっており、切替を先に済ませると公開直後の履歴が空になります。

検出できない — 書式宣言の 2 つの原理的な限界

`SHM_DECLARE_LAYOUT()` はメンバの書き漏らしをコンパイルエラーにしますが、`offsetof` と `sizeof` から判別できない場合が 2 つあります。

- 書き漏らしたメンバが「どのみち必要なパディング」に収まる場合。通常の型では隙間は最大 `alignof(T)-1` バイトなので、入るのは小さなメンバに限られます。

- 型に `alignas` を付けると `sizeof` が切り上がるため、末尾で窓が広がります。 `alignas(32)` なら 31 バイトまで見逃します。

またビットフィールドを含む型は宣言できません（`offsetof` も `sizeof` も使えないため）。配置が実装定義でプロセス間の合意にできないので、共有メモリに載せる型では使わないでください。

未着手 — マージ後に予定している作業

- **SHM_REQUIRE_LAYOUT の既定 ON 化** — 147 種類の型を一斉に必須化すると複数リポジトリのビルドが同時に止まるため、パッケージ単位で移行してから既定を変える方針です。
- **GitHub Pages のソース切り替え** — リポジトリ設定で Source を「GitHub Actions」にする作業が残っています。PR では生成と検査だけ走り配信しないので、切り替え前に緑を確認できます。
- **R05 の Low 指摘のうち塞げないもの** — 上記 2 件がそれにあたります。判断理由は `review/r05-remediation.md` に、塞ぐ 3 案がそれぞれ何を壊すかまで記録しています。

参照

- `review/` — レビュー記録 12 本（R01～R05 の所見と対応、R06 のリファクタリング記録）
- `manual/pitfalls_jp.md` — 利用者向けの注意点 12 項目
- `manual/spec_jp.md` — 仕様

本文中の数値・挙動はすべて手元で実測または再現したものです。未検証の事項は「未着手」として明示しています。